DM_Assignment 2

414551017 陳冠甫

GitHub：https://github.com/Patrickchen2000/DM2026-Assignment-2

1. K-fold Cross-Validation (15 points):

1(a). Implement 5-fold cross-validation on the training data

4x4 Average Accuracy Table (Training Data, 5-Fold CV)

Regularization (Lambda) \ Learning Rate	0.005	0.010	0.100	0.500
1.0	0.711429	0.725714	0.734286	0.734286
2.0	0.717143	0.740000	0.734286	0.725714
4.0	0.722857	0.737143	0.740000	0.542857
8.0	0.725714	0.725714	0.725714	0.462857

In this experiment, we applied 5-fold cross-validation on the training data to determine the optimal hyperparameters for the model with L2 regularization.

A total of **16** combinations were evaluated, with learning rates {0.005, 0.01, 0.1, 0.5} and regularization parameters λ {1.0, 2.0, 4.0, 8.0}. **For each combination, the average accuracy across the five folds was computed.** The results are summarized in a **4×4 table**, and the best-performing hyperparameter combination was selected accordingly.

1(b). Following 1(a), **select the top two hyperparameter settings** based on the training data.

	Regularization (Lambda)	Learning Rate	mean_cv_accuracy
1	2.0	0.01	0.74
2	4.0	0.10	0.74

Following the cross-validation results in 1(a), the top two hyperparameter settings with the highest average accuracy were selected. These are **λ = 2.0 with learning rate = 0.01** and **λ = 4.0 with learning rate = 0.1**, both achieving an average cross-validation accuracy of 0.74.

The model was subsequently retrained on the entire training dataset using each selected hyperparameter setting and evaluated on the testing dataset using the evaluate_binary_classifier function.

```
===== Top 1 Setting =====
Learning Rate = 0.01
Regularization Lambda = 2.0
5-Fold CV Average Accuracy = 0.7400
Model Evaluation
Accuracy  : 0.7533
Precision : 0.7407
Recall    : 0.7895
F1-score  : 0.7643
```

For the first setting ($\lambda = 2.0$, learning rate = 0.01), the model achieved an accuracy of 0.7533, precision of 0.7407, recall of 0.7895, and F1-score of 0.7643.

```
===== Top 2 Setting =====
Learning Rate = 0.1
Regularization Lambda = 4.0
5-Fold CV Average Accuracy = 0.7400
Model Evaluation
Accuracy  : 0.7600
Precision : 0.7381
Recall    : 0.8158
F1-score  : 0.7750
```

For the second setting ($\lambda = 4.0$, learning rate = 0.1), the model achieved an accuracy of 0.7600, precision of 0.7381, recall of 0.8158, and F1-score of 0.7750.

(c). Describe your observations in 1(a) and 1(b).

From the results in 1(a), **we observe that both the learning rate and the regularization parameter λ have a significant impact on model performance.**
Specifically, moderate learning rates (0.01 and 0.1) consistently achieve higher average accuracy compared to very small or large values. When the learning rate is too large (e.g., 0.5), the performance drops noticeably, which suggests unstable training or poor convergence.
Similarly, increasing λ from 1.0 to 4.0 slightly improves performance, indicating that L2 regularization helps reduce overfitting; however, when

λ becomes too large (e.g., 8.0), the accuracy decreases, suggesting underfitting due to overly strong regularization.

From the results in 1(b), both selected hyperparameter settings achieve similar performance on the testing data, indicating that the model generalizes well across these configurations. Although both settings have the same cross-validation accuracy, the combination λ = 4.0 and learning rate = 0.1 achieves slightly higher testing accuracy and F1-score, mainly due to a higher recall. This suggests that this setting is better at identifying positive samples, though it comes with a slight trade-off in precision.

Overall, the results demonstrate that carefully balancing the learning rate and regularization strength is crucial for achieving good performance, and that **cross-validation is effective in identifying hyperparameter settings** that generalize well to unseen data.

2. SVM (15 points):

2(a). Report the **accuracy and F1-score** on training, validation and testing data, respectively.

```
Train Accuracy: 0.9450
Train F1-score: 0.9445
------
Validation Accuracy: 0.9450
Validation F1-score: 0.9448
------
Test Accuracy: 0.9575
Test F1-score: 0.9573
------
```

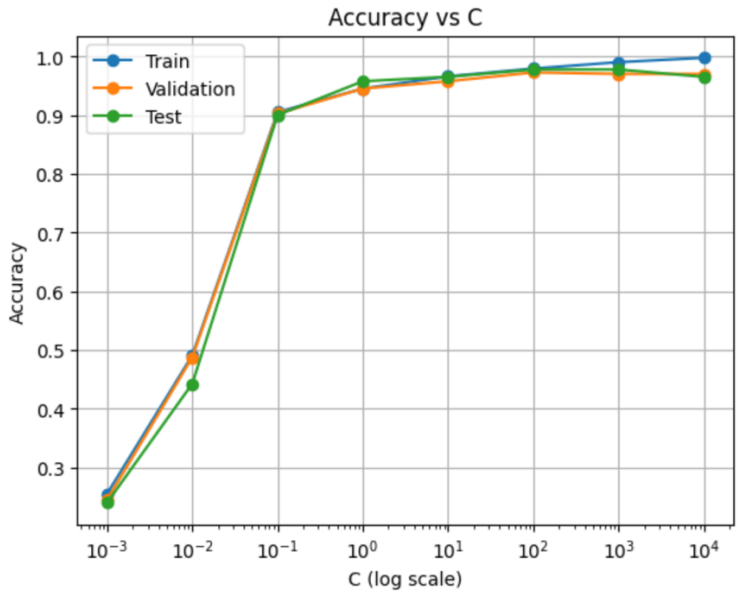
- Training Set:
  The model achieved an accuracy of 0.9450 and an F1-score of 0.9445, indicating that it **fits the training data well.**
- Validation Set:
  On the validation set, the accuracy is 0.9450 and the F1-score is 0.9448, which are **very close** to the training results. This suggests that the model **generalizes well and does not suffer from overfitting.**
- Testing Set:

The model achieved an accuracy of 0.9575 and an F1-score of 0.9573 on the test set. These **slightly higher values further confirm that the model performs consistently and robustly on unseen data.**

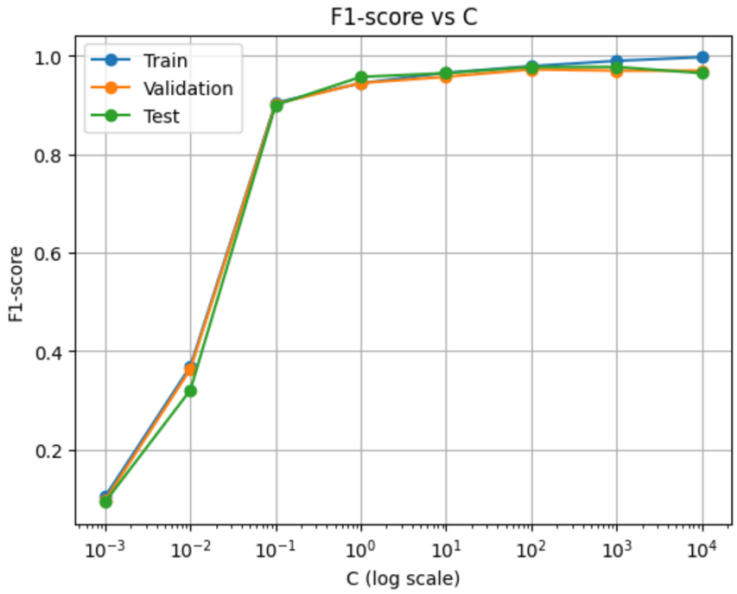
Overall, the SVM classifier demonstrates strong and stable performance across training, validation, and testing datasets. The similarity of metrics across splits indicates good generalization ability, and there is no clear sign of overfitting or underfitting.

2(b). Explore **different values** of the regularization parameter **C** = {0.001, 0.01, 0.1, … 100, 1000, 10000}.

Accuracy with Respect to C



F1-score with Respect to C



From the figures, we can observe how the performance of the SVM model changes as the regularization parameter C varies.

- When C is very small (e.g., 0.001 to 0.01), both accuracy and F1-score are low across the training, validation, and test sets. This indicates that the model suffers from underfitting, as strong regularization prevents it from capturing the underlying patterns in the data.
- As C increases to moderate values (e.g., 0.1 to 10), the performance improves significantly. In this range, the accuracy and F1-score exceed 0.9, and the results on the training, validation, and test sets are very close to each other. This suggests that the model achieves good generalization without overfitting.
- However, when C becomes very large (e.g., 100 to 10000), the training performance continues to increase and approaches 1.0, while the validation and test performance plateau or slightly decrease. This behavior indicates the onset of overfitting, where the model fits the training data too closely but does not generalize well to unseen data.

Overall, the model achieves the best performance at moderate values of C (approximately between 1 and 100). Smaller values lead to underfitting, while larger values result in overfitting. **Therefore, selecting an appropriate value of C is crucial for balancing bias and variance in the SVM model.**

	C	Train Acc	Val Acc	Test Acc	Train F1	Val F1	Test F1
0	0.001	0.255000	0.2450	0.2400	0.103625	0.096426	0.092903
1	0.010	0.490833	0.4875	0.4425	0.368342	0.362820	0.320378
2	0.100	0.905000	0.9025	0.9000	0.904724	0.902429	0.899996
3	1.000	0.945000	0.9450	0.9575	0.944525	0.944814	0.957272
4	10.000	0.965833	0.9575	0.9650	0.965776	0.957543	0.964960
5	100.000	0.979167	0.9725	0.9775	0.979170	0.972611	0.977554
6	1000.000	0.990000	0.9700	0.9775	0.989992	0.969976	0.977538
7	10000.000	0.997500	0.9700	0.9650	0.997500	0.970026	0.964963

```
Best C: 100
Best Validation Accuracy: 0.9725
```

2(c). Following 2(b), based on your observations, determine which model (which value of C) **provides the best generalization performance**, and explain your reasoning.

Based on the observations from 2(b), the model with C = 100 provides the best generalization performance.
From the table, **when C=100, the validation accuracy reaches its highest value (0.9725)**, and the corresponding F1-score (0.9726) is also very high. In addition, the **test performance (accuracy = 0.9775, F1-score = 0.9776) is strong and consistent with the validation results**, indicating that the model generalizes well to unseen data.

Therefore, C=100 achieves the best balance between bias and variance, making it the optimal choice for this task.

3. Association Rule Mining (15 points):

3(a). Please list all frequent patterns showing **support ≥ 0.3** using FP-growth.

	support	itemsets
2	0.682	frozenset({ram_medium})
3	0.416	frozenset({px_width_medium})
5	0.414	frozenset({battery_power_medium})
4	0.412	frozenset({int_memory_medium})
7	0.318	frozenset({battery_power_medium, ram_medium})
0	0.316	frozenset({int_memory_low})
1	0.308	frozenset({battery_power_low})
6	0.306	frozenset({ram_medium, px_width_medium})

The FP-growth algorithm was applied to the dataset with a minimum support threshold of 0.3. All frequent patterns satisfying this condition were extracted.

The results include both single-item itemsets and multi-item itemsets. For example, {ram_medium} has the highest support (0.682), while other frequent itemsets such as {int_memory_low}, {battery_power_low}, and {px_width_medium, ram_medium} also meet the minimum support requirement. **These patterns indicate commonly occurring attribute combinations in the dataset.**

3(b). Following 3(a), please list all the association rules where **support ≥ 0.3, confidence ≥ 0.4 and lift ≥ 0.8**

	antecedents	consequents	support	confidence	lift
2	battery_power_medium	ram_medium	0.318	0.768116	1.126270
0	px_width_medium	ram_medium	0.306	0.735577	1.078559
3	ram_medium	battery_power_medium	0.318	0.466276	1.126270
1	ram_medium	px_width_medium	0.306	0.448680	1.078559

```
{'battery_power_medium'} -> {'ram_medium'}
support = 0.318, confidence = 0.768, lift = 1.126
--------------------------------------------------
{'px_width_medium'} -> {'ram_medium'}
support = 0.306, confidence = 0.736, lift = 1.079
--------------------------------------------------
{'ram_medium'} -> {'battery_power_medium'}
support = 0.318, confidence = 0.466, lift = 1.126
--------------------------------------------------
{'ram_medium'} -> {'px_width_medium'}
support = 0.306, confidence = 0.449, lift = 1.079
--------------------------------------------------
```

Based on the frequent itemsets obtained in 3(a), association rules were generated and filtered using the thresholds of support ≥ 0.3, confidence ≥ 0.4, and lift ≥ 0.8. As a result, **four association rules satisfy all the specified criteria, all of which involve ram_medium and its relationships with other attributes.**

Specifically, the rules {battery_power_medium} → {ram_medium} and {px_width_medium} → {ram_medium} have relatively high confidence values (0.768 and 0.736, respectively), **indicating a strong likelihood of ram_medium occurring when these conditions are met.** Additionally, the reverse rules {ram_medium} → {battery_power_medium} and {ram_medium} → {px_width_medium} also meet the thresholds, suggesting mutual associations among these attributes.

Overall, all the selected rules have **lift values greater than 1, indicating positive correlations between the variables and demonstrating meaningful relationships within the dataset.**

3(c). Please describe your observations in 3(a) and 3(b).

From the results of 3(a), it can be observed that ram_medium has the highest support, indicating that it appears most frequently in the dataset. It also frequently co-occurs with other attributes, such as battery_power_medium and px_width_medium, forming multi-item frequent patterns. This suggests that ram_medium plays a key role in the dataset.

From the association rules in 3(b), it can be seen that several rules involve ram_medium, and all of them satisfy the confidence threshold. This indicates that **when certain conditions are met, ram_medium is likely to occur.** In addition, all rules have lift values greater than 1, which implies positive correlations among these attributes rather than random co-occurrence.

Overall, the relationships in the dataset **are mainly centered around ram_medium,** indicating that it has strong and stable associations with other attributes and **effectively reflects the underlying structure of the data.**

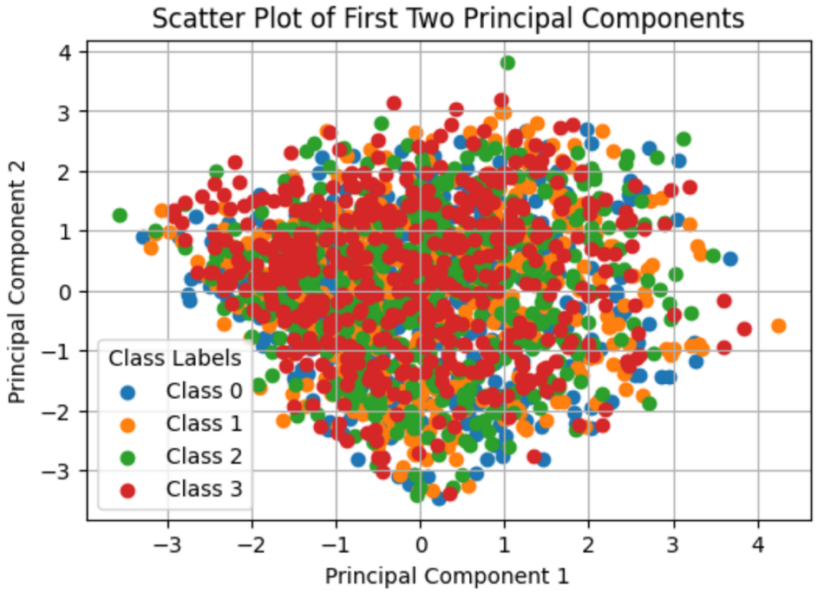
4. PCA and K-Means (20 points):

4(a). Standardize the features using z-score standardization.

```
Features shape: (2000, 20)
Labels shape: (2000,)
[[-0.90259726 -0.9900495   0.83077942 -1.01918398 -0.76249466 -1.04396559
  -1.38064353  0.34073951  1.34924881 -1.10197128 -1.3057501  -1.40894856
  -1.14678403  0.39170341 -0.78498329  0.2831028   1.46249332 -1.78686097
  -1.00601811  0.98609664]
 [-0.49513857  1.0100505  -1.2530642   0.98117712 -0.99289039  0.95788598
   1.15502422  0.68754816 -0.12005944 -0.66476784 -0.64598879  0.58577791
   1.70446468  0.46731702  1.11426556 -0.63531667 -0.73426721  0.55964063
   0.99401789 -1.01409939]
 [-1.5376865   1.0100505  -1.2530642   0.98117712 -0.53209893  0.95788598
   0.49354568  1.38116548  0.13424391  0.20963905 -0.64598879  1.39268422
   1.07496821  0.44149774 -0.31017108 -0.86492153 -0.36814045  0.55964063
   0.99401789 -1.01409939]
 [-1.41931861  1.0100505   1.19851653 -1.01918398 -0.99289039 -1.04396559
  -1.2152739   1.03435682 -0.26133908  0.6468425  -0.15116781  1.28674959
   1.23697097  0.59456919  0.87685946  0.51270767 -0.0020137   0.55964063
  -1.00601811 -1.01409939]
 [ 1.32590586  1.0100505  -0.39501094 -1.01918398  2.00225408  0.95788598
   0.65891532  0.34073951  0.0212202  -1.10197128  0.67353383  1.26871816
  -0.09145172 -0.65766599 -1.0223894  -0.86492153  0.73023981  0.55964063
   0.99401789 -1.01409939]]
```

The dataset **was first split into features (X) and labels (y)**, where X contains all input variables and y represents the target variable (price range). After that, the **features were standardized** using z-score standardization with StandardScaler from sklearn.preprocessing.
This method transforms each feature by subtracting its mean and dividing by its standard deviation, resulting in a dataset where all features have a mean of 0 and a standard deviation of 1. This standardization helps **ensure that all features are on the same scale, improving the performance and stability of machine learning models.**

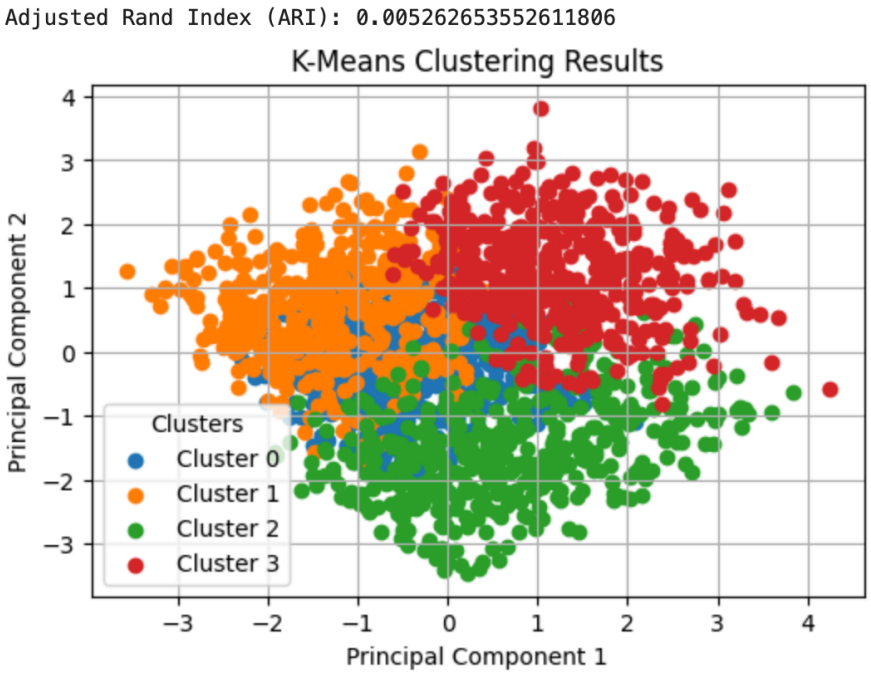
4(b). Following 4(a), **project the features onto 2 dimensions using PCA**.



Following the standardization in 4(a), Principal Component Analysis **(PCA) was applied to project the features onto two dimensions, and a scatter plot of the first two principal components was generated, as shown in the figure.** Each point represents a data sample, and different colors indicate different classes (Class 0–3).

It can be observed that most data points are **concentrated near the center and that there is significant overlap among the classes, with no clear separation or distinct clusters.** This indicates that although the first two principal components capture the main variance in the data, they are not sufficient to clearly distinguish between different classes, **suggesting that important discriminative information may exist in higher dimensions.**

4(c). Following 4(a), apply K-means to cluster the data samples into 4 clusters **using all features.**

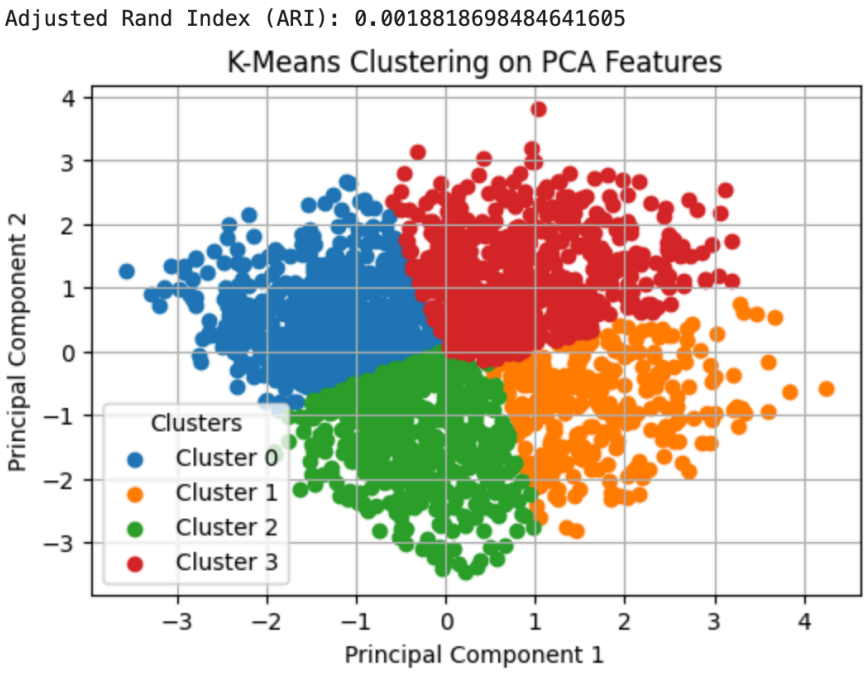


Following the standardization in 4(a), K-means clustering (k = 4) was applied to the dataset using all features. The clustering results were then projected onto two dimensions using PCA and visualized in a scatter plot, where different colors represent different clusters (Cluster 0–3). As shown in the figure, although some clusters exhibit partial spatial grouping, there is still significant overlap among them, especially near the center, **indicating that the cluster boundaries are not clearly defined.**

For performance evaluation, the Adjusted Rand Index (ARI) was used, yielding a value of 0.0052. Since ARI ranges from -1 to 1, where values closer to 1 indicate strong agreement with the true labels and values near 0 indicate random clustering, this result suggests that the clustering is almost equivalent to random assignment.

Overall, both the visualization and the ARI score indicate that K-means fails to effectively separate the data into meaningful groups, likely due to overlapping class distributions or the lack of clear cluster structure in the feature space.

4(d). Following 4(b), apply K-means to cluster the data samples into 4 clusters **using the 2-dimensional features obtained from PCA.**



Following 4(b), the data was first reduced to two dimensions using PCA, and K-means clustering (k = 4) was then applied using these 2D features. The clustering results are shown in the scatter plot, where different colors represent different clusters (Cluster 0–3). It can be observed that the data is partitioned into several wedge-like regions in the 2D space, indicating that K-means mainly separates the data based on distance from the cluster centers.

However, this clustering structure **does not align well with the true class labels.** Although the clusters appear more spatially separated compared to using the original features, **they do not reflect the actual underlying class distribution.**

In terms of performance, the Adjusted Rand Index (ARI) is 0.0018. Since **this value is very close to 0, it indicates that the clustering result is almost equivalent to random assignment and has very low agreement with the true labels.**

Compared to 4(c), where clustering was performed on the original features (ARI ≈ 0.005), the performance **slightly decreases after applying PCA.**

Overall, while **PCA simplifies the data and improves visualization,** it may also remove important discriminative information, leading to no improvement or even slight degradation in K-means clustering performance.

4(e). Please describe your observations in 4(c) and 4(d).

In 4(c), K-means clustering was applied using all original features, while in 4(d), clustering was performed on the 2D features obtained from PCA.

From the results, **both approaches show poor clustering performance,** as **indicated by the very low Adjusted Rand Index (ARI) values** (approximately 0.005 in 4(c) and 0.001 in 4(d)), suggesting that the clustering results **are close to random assignment** and **do not align well with the true labels.**

From the visualizations, the clusters in 4(c) show significant overlap, indicating that the original feature space does not have a clear cluster structure. **In 4(d), although the PCA projection makes the clusters appear more spatially separated in the 2D plot, the clustering is mainly based on geometric partitioning (e.g., wedge-like regions) rather than true class distinctions.**

Overall, applying PCA before K-means does not improve clustering performance and may even slightly degrade it, as dimensionality reduction can lead to the loss of important discriminative information. Both results suggest that the dataset does not exhibit strong natural clustering aligned with the true class labels.

5. Enhancing K-means Clustering with Association Rule Mining (35 points):

As the baseline method, the original K-means algorithm was applied directly to all features in the dataset with the number of clusters set to K=4, corresponding to the four mobile price categories.
To ensure fair evaluation and reduce the influence of initialization randomness, the experiments were repeated using five different random seeds : [0,10,42,100,999]. The clustering performance was evaluated using **accuracy, precision, recall, and F1-score, and the final results were averaged across all seeds.**

The baseline achieved an average accuracy of approximately 29.62%, with similar values for precision, recall, and F1-score. These results indicate that the standard K-means algorithm has limited capability in capturing the underlying structure of the dataset when relying only on distance-based clustering. Therefore, additional techniques such as association rule mining may help discover hidden relationships among features and further improve clustering performance.

Baseline Results

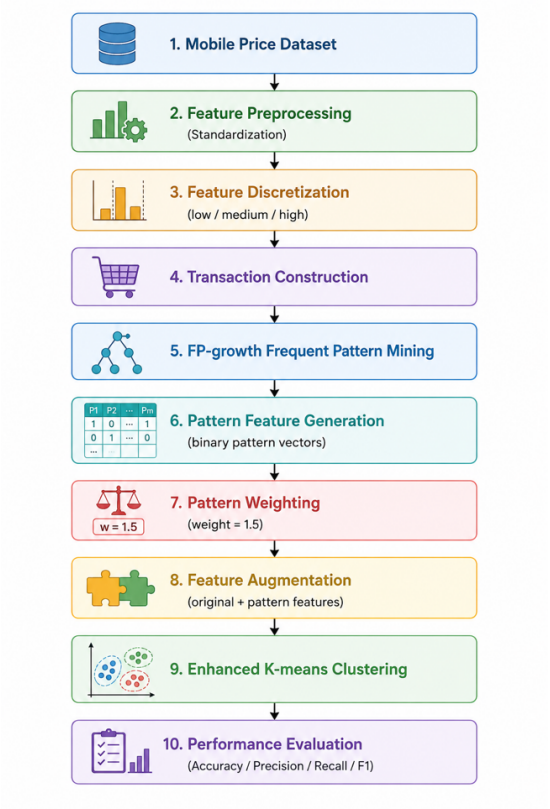
Results for each seed:

	seed	accuracy	precision	recall	f1
0	0	0.2945	0.294042	0.2945	0.292637
1	10	0.2955	0.295336	0.2955	0.293648
2	42	0.2970	0.296623	0.2970	0.295035
3	100	0.2955	0.294995	0.2955	0.293510
4	999	0.2985	0.298367	0.2985	0.295579

Proposed Method:

FP-growth-based Pattern Feature Augmentation for K-means

Framework diagram



1. Framework Overview

This study proposes an FP-growth-enhanced K-means framework to improve clustering performance by incorporating association-rule-based pattern features into the clustering process. The proposed method aims to discover hidden relationships among important mobile phone attributes and transform these relationships into informative representations for clustering.

The overall framework consists of the following stages:

1. Feature selection and preprocessing
2. Feature discretization and transaction construction
3. Frequent pattern mining using FP-growth
4. Pattern feature generation and weighting
5. Enhanced K-means clustering
6. Performance evaluation and comparison

2. Feature Selection and Preprocessing

In the proposed method, FP-growth was incorporated into K-means clustering to enhance the clustering performance by discovering hidden relationships among important mobile phone features. Specifically, four representative features (ram, int_memory, px_width, and battery_power) **were selected as the source for association rule mining because these attributes are highly related to mobile phone price categories.**

Since association rule mining requires categorical transaction-style data, each numerical feature was first discretized into three levels: low, medium, and high, using a (30%-40%-30%) interval strategy. The discretized values were then transformed into transaction items such as ram_high and battery_power_low.

3. Frequent Pattern Mining Using FP-growth

After preprocessing, the FP-growth algorithm was applied to extract frequent itemsets with a minimum support threshold of 0.3. These frequent patterns represent commonly co-occurring feature combinations within the dataset. Instead of directly using association rules for classification, the discovered frequent patterns were converted into additional binary pattern features, where each feature indicates whether a data sample satisfies a specific frequent pattern.

4. Pattern Feature Augmentation

The generated pattern features were subsequently standardized and multiplied by a weighting factor of 1.5 to emphasize their contribution. Finally, these weighted pattern features were concatenated with the original standardized features to construct an enhanced feature space for clustering.

5. Enhanced K-means Clustering

K-means clustering was then performed on the enhanced feature space using the same number of clusters and the same set of random seeds as the baseline method for fair comparison.

6. Inspiration and Design Rationale

The main inspiration behind this design comes from association-rule-based feature engineering, which aims to transform hidden co-occurrence relationships among attributes into informative representations for machine learning tasks.

Compared with the original K-means algorithm that relies purely on distance-based similarity, the proposed method allows the clustering process to consider both numerical similarity and frequent attribute interaction patterns, thereby improving the capability of identifying more meaningful cluster structures.

7. Experimental Results

7.1 Results of the Proposed FP-growth-enhanced K-means

	seed	method	accuracy	precision	recall	f1
0	0	FP-growth K-means	0.3630	0.427274	0.3630	0.375350
1	10	FP-growth K-means	0.3630	0.427274	0.3630	0.375350
2	42	FP-growth K-means	0.4495	0.469815	0.4495	0.457241
3	100	FP-growth K-means	0.3630	0.427274	0.3630	0.375350
4	999	FP-growth K-means	0.3630	0.427274	0.3630	0.375350

The experimental results demonstrate that the proposed FP-growth-enhanced K-means approach outperformed the original K-means baseline in clustering performance. Using the same set of random seeds [0,10,42,100,999], the proposed approach consistently achieved higher evaluation scores across all metrics. These findings indicate that incorporating frequent pattern information into the clustering process can effectively enhance cluster quality.

7.2 Comparison with Original K-means

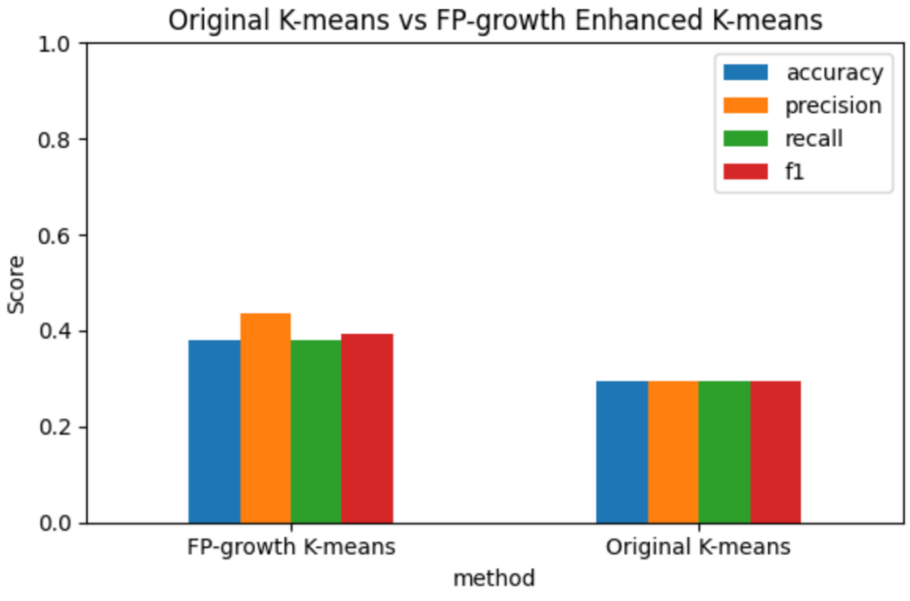
```
Average comparison:
                   accuracy  precision  recall        f1
method
FP-growth K-means    0.3803   0.435782  0.3803  0.391728
Original K-means     0.2962   0.295873  0.2962  0.294082
```

The average experimental results further validate the effectiveness of the proposed FP-growth-enhanced K-means method. Compared with the original K-means algorithm, the proposed approach achieved noticeable improvements across all evaluation metrics. Specifically, accuracy increased from 29.62% to 38.03%, precision improved significantly from 29.59% to 43.58%, recall increased from 29.62% to 38.03%, and the F1-score improved from 29.41% to 39.17%.

These results demonstrate that incorporating FP-growth-based pattern features can effectively enhance clustering performance and improve the cluster separation capability of K-means beyond the original baseline method.

8. Conclusion:

From the comparison figure, the proposed FP-growth-enhanced K-means consistently achieves higher scores than the original K-means in all evaluation metrics. The improvement is especially significant in precision, indicating that the proposed method reduces incorrect cluster assignments more effectively than the baseline approach.



Another important observation is that the enhanced method maintains relatively stable performance across different random seeds, suggesting that the additional pattern features help reduce the sensitivity of K-means to random initialization. In contrast, the original K-means remains limited by its reliance on distance-based similarity alone.

The results also suggest that incorporating association-rule-derived features provides additional structural information that is not fully captured by the original numerical features. Therefore, the clustering boundaries become clearer, leading to better overall clustering quality.